

Universidad del Valle de Guatemala

Programación Orientada a Objetos

Segundo Ciclo 2026



## Ejercicio #4

Erwin Tomás 26234

Sección 20

27/09/2026

## **Análisis**

### **Información que se debe conservar:**

Datos de los vehículos, como placa, marca, modelo, tarifa y estado

Para automóvil debe guardar la capacidad y transmisión

Para motocicleta el cilindraje

Para camioneta de carga capacidad de transporte

Dinero acumulado por lo alquileres que están confirmados

### **Operaciones que debe permitir:**

Registrar vehículos

Consultar datos de un vehículo

Cotizar el precio de alquiler de un vehículo

Conformar alquiler

Permitir devoluciones

Consultar reportes al final del día, que incluya total de vehículos, disponibilidad y dinero acumulado total

### **Condiciones que determinan si las operaciones son válidas:**

No puede haber dos vehículos con la misma placa

No se puede alquilar un vehículo que esté ocupado

Si se cancela un alquiler el vehículo pasa a estar disponible y no se registra el cobro

Las cotizaciones no deben afectar el monto total acumulado de la empresa

### **Clases:**

#### **Vehículo:**

Tiene los atributos y comportamientos de todos los vehículos y tiene la estructura general para calcular las tarifas.

Atributos:

placa: String, private

marca: String, protected

modelo: String, protected

tarifaDiaria: double, protected

disponible: Boolean, protected

diasAlquiler: int, protected

Métodos:

calcularCosto (dias: int): double

obtenerDetalles (): String

getPlaca (): String

estaDisponible (): Boolean

setDisponibilidad (estado: boolean): void

### **Automóvil:**

Gestiona a los automóviles, como sus atributos específicos y el cobro.

Atributos:

pasajeros: int, private

esAutomatico: boolean, private

Métodos:

Automovil (placa: String, marca: String, modelo: String, tarifaDiaria: double, pasajeros: int, esAutomatico: boolean)

calcularCosto (días: int): double

### **Motocicleta:**

Gestiona las motocicletas, su cilindraje y las tarifas

Atributos:

Cilindraje: int, private

Métodos:

Motocicleta (placa: String, marca: String, modelo: String, tarifaDiaria: double, cilindraje: int)

calcularCosto (días: int) double

### **CamionetaCarga**

Gestiona las camionetas, su capacidad de carga y tarifa, incluyendo el cargo por tonelada

Atributos:

capacidadToneladas: double, private

Métodos:

CamionetaCarga (placa: String, marca: String, modelo: String, tarifaDiaria: double, capacidadToneladas: double)

calcularCosto (días: int): double

### **GestorAlquileres**

Administra los vehículos, procesa las cotizaciones, valida y confirma alquileres, registra devoluciones y genera los reportes del final del día.

Atributos:

vehículos: List, private

ingresosAcumulados: double, private

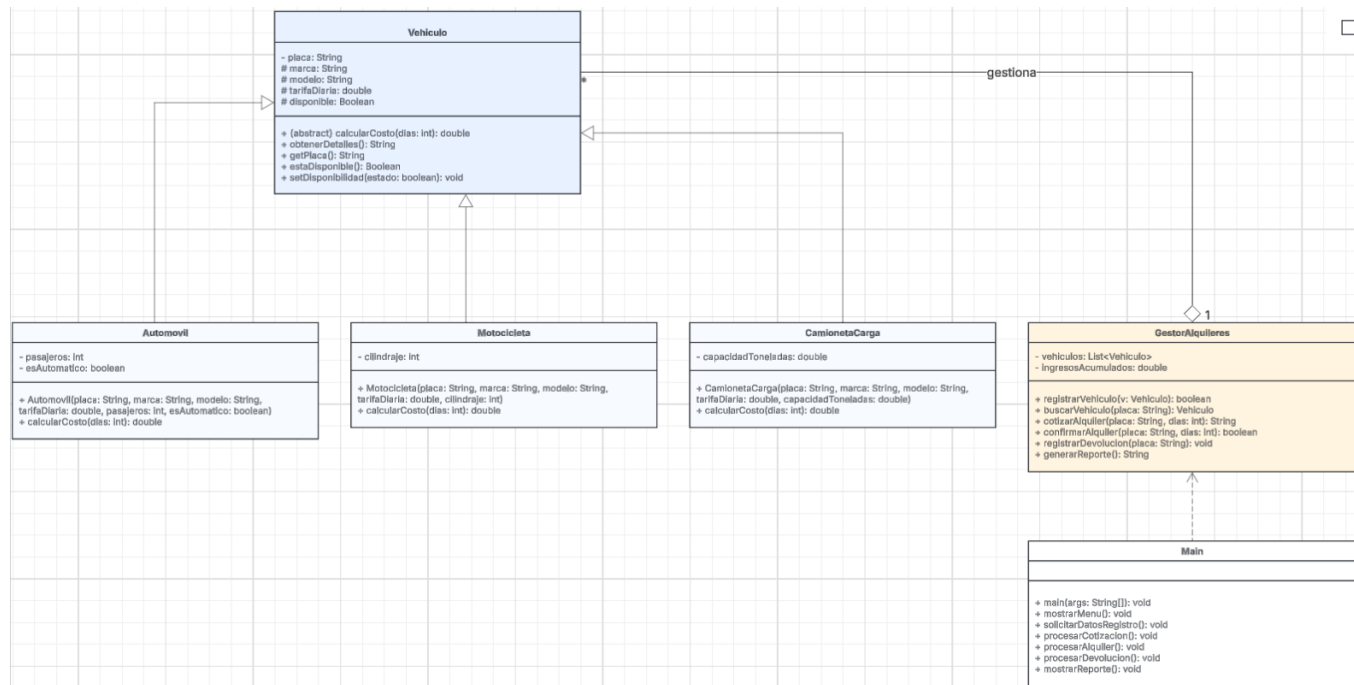
Métodos:

registrarVehiculo (v: vehículo): boolean  
 buscarVehiculo (placa: String): Vehículo  
 cotizarAlquiler (placa: String, días: int): String  
 confirmarAlquiler (placa: String, días: int): boolean  
 registrarDevolución (placa: String): void  
 generarReporte (): String

Los elementos que se comparten entre vehículos, como la placa, marca, modelo, tarifa y estado están en la clase Vehículo, lo que evita que se repita código. Las diferencias en los cobros los resolveré con overload del método calcularCosto. Las decisiones de cobro están en los métodos específicos de las clases hijas. Para hacer un alquiler la clase GestorAlquileres es la que actúa como medio entre las diferentes clases, busca el vehículo por su placa y valida el estado de disponibilidad.

Para nuevas categorías de vehículos no es necesario modificar GestorAlquileres ni las demás clases. Solo se debe crear otra clase hija que extienda de vehículo y generar su método de calcularCosto.

## Diagrama UML



[https://lucid.app/lucidchart/f768e20d-1826-401a-9da2-cc423a0a51ba/edit?viewport\\_loc=-197%2C-8%2C3082%2C1576%2CHWEp-vi-RSFO&invitationId=inv\\_ed47f36c-a3bd-488a-b1b4-1e617d7ca2f2](https://lucid.app/lucidchart/f768e20d-1826-401a-9da2-cc423a0a51ba/edit?viewport_loc=-197%2C-8%2C3082%2C1576%2CHWEp-vi-RSFO&invitationId=inv_ed47f36c-a3bd-488a-b1b4-1e617d7ca2f2)

